DAA Complete Viva Preparation

Unit I – Algorithms and Problem Solving

Basic Algorithm Questions

Q1. What is an Algorithm?

Ans. An algorithm is a finite sequence of well-defined steps used to solve a particular problem.

Q2. What are the characteristics of an algorithm?

Ans. Input, Output, Definiteness, Finiteness, and Effectiveness are the five main characteristics.

Q3. Why are algorithms important in computing?

Ans. Algorithms help computers solve problems correctly and efficiently using minimum time and memory.

Q4. What does "Algorithms as Technology" mean?

Ans. An efficient algorithm can improve performance like faster hardware, often without changing the computer.

Q5. What is Algorithm Design?

Ans. Algorithm Design is the process of creating step-by-step instructions to solve a problem efficiently.

Q6. What is Algorithm Correctness?

Ans. An algorithm is correct if it produces the expected output for every valid input and terminates.

Q7. How do we verify algorithm correctness?

Ans. We use loop invariants, mathematical induction, and correctness proofs. Testing helps find errors but cannot prove correctness for every input.

Q8. What is a Loop Invariant?

Ans. A loop invariant is a condition that remains true before and after every iteration of a loop.

Q9. What is an Iterative Algorithm?

Ans. An iterative algorithm repeats instructions using loops such as for and while.

Q10. What are the main issues in iterative algorithm design?

Ans. Ensuring correct initialization, loop condition, progress, termination, and correctness.

Problem-Solving Principles

Q11. What are problem-solving strategies?

Ans. Common strategies are Brute Force, Divide and Conquer, Greedy, Dynamic Programming, and Backtracking.

Q12. What is Brute Force?

Ans. Brute Force tries all possible solutions or straightforward possibilities to find a valid or optimal answer.

Q13. What is Problem Classification?

Ans. Problem Classification groups problems by their nature, such as searching, sorting, optimization, decision, and counting problems.

Q14. What is Logarithmic Time Complexity?

Ans. O(log n) means the problem size is reduced by a constant factor at each step. Example: Binary Search.

Q15. What is Linear Time Complexity?

Ans. O(n) means the running time grows proportionally with input size. Example: Linear Search.

Tower of Hanoi

Q16. What is Tower of Hanoi?

Ans. Tower of Hanoi is a recursive problem where disks are moved between three rods without placing a larger disk on a smaller disk.

Q17. Explain Tower of Hanoi Algorithm.

Ans.

1. Move n−1 disks to the auxiliary rod.
2. Move the largest disk to the destination.
3. Move n−1 disks onto the largest disk.

Q18. What is the minimum number of moves?

Ans. For n disks, the minimum number of moves is $2^n - 1$.

Q19. What is the complexity of Tower of Hanoi?

Ans. Time $O(2^n)$, Space O(n) due to the recursive call stack.

Unit II – Analysis of Algorithms and Complexity Theory

Algorithm Analysis

Q20. What is Analysis of Algorithms?

Ans. It is the process of measuring an algorithm's time and space requirements based on input size.

Q21. What is Input Size?

Ans. Input Size (n) represents the amount of data processed by an algorithm.

Q22. What is Best Case?

Ans. Best Case is the minimum running time for any input of a given size.

Q23. What is Worst Case?

Ans. Worst Case is the maximum running time for any input of a given size.

Q24. What is Average Case?

Ans. Average Case is the expected running time over inputs according to a specified probability distribution.

Q25. What is a Dominant Operator?

Ans. A dominant operator is a key operation whose execution count is used to estimate running time, such as comparisons in sorting.

Q26. What is Growth Rate?

Ans. Growth Rate describes how an algorithm's running time increases as the input size increases.

Asymptotic Notations

Q27. What is Big O Notation?

Ans. Big O (O) represents an asymptotic upper bound on the growth of a function.

Q28. What is Omega Notation?

Ans. Omega (Ω) represents an asymptotic lower bound on the growth of a function.

Q29. What is Theta Notation?

Ans. Theta (Θ) represents a tight bound, meaning both upper and lower bounds.

Q30. What is Little o Notation?

Ans. Little o (o) represents a strict asymptotic upper bound.

Q31. What is Little Omega Notation?

Ans. Little Omega (ω) represents a strict asymptotic lower bound.

Q32. Which is better: $O(n)$ or $O(n^2)$?

Ans. $O(n)$ generally scales better because its running time grows more slowly for large inputs.

Complexity Theory

Q33. What is a Polynomial-Time Algorithm?

Ans. An algorithm whose running time is bounded by $O(n^k)$ for some constant k.

Q34. What is a Non-Polynomial-Time Algorithm?

Ans. An algorithm whose running time cannot be bounded by any polynomial in input size, such as $O(2^n)$.

Q35. What is a Deterministic Algorithm?

Ans. A deterministic algorithm follows uniquely defined steps for a given input and state.

Q36. What is a Non-Deterministic Algorithm?

Ans. A non-deterministic algorithm is a theoretical model that can explore multiple possible computation choices.

Q37. What is Class P?

Ans. P contains decision problems solvable in polynomial time using a deterministic algorithm.

Q38. What is Class NP?

Ans. NP contains decision problems whose proposed solutions can be verified in polynomial time.

Q39. What is an NP-Complete Problem?

Ans. An NP-Complete problem belongs to NP and is at least as hard as every problem in NP.

Q40. What is an NP-Hard Problem?

Ans. An NP-Hard problem is at least as hard as every problem in NP, but it need not belong to NP.

Q41. What is Polynomial-Time Reduction?

Ans. It is converting one problem into another in polynomial time, while preserving the answer.

Q42. What is the Vertex Cover Problem?

Ans. Find a set of vertices such that every edge has at least one endpoint in the selected set.

Q43. What is 3-SAT?

Ans. 3-SAT checks whether a Boolean formula in CNF, with three literals per clause, has a satisfying assignment.

Q44. What is the Hamiltonian Cycle Problem?

Ans. It checks whether a graph contains a cycle that visits every vertex exactly once and returns to the starting vertex.

Q45. Is the Hamiltonian Cycle Problem NP-Hard?

Ans. Yes. Its decision version is NP-Complete, so it is also NP-Hard.

Unit III – Greedy and Dynamic Programming

Greedy Strategy

Q46. What is Greedy Method?

Ans. Greedy Method makes the best available local choice at each step to build a solution.

Q47. What is Greedy Choice Property?

Ans. It means an optimal solution can be reached by making locally optimal choices.

Q48. What is Control Abstraction?

Ans. Control Abstraction is a general algorithm structure that shows the main steps without implementation details.

Q49. What is Fractional Knapsack?

Ans. Fractional Knapsack allows selecting fractions of items to maximize profit within the given capacity.

Q50. How does Fractional Knapsack work?

Ans.

1. Calculate profit/weight ratio.
2. Sort items by decreasing ratio.
3. Select full items while possible.
4. Take a fraction of the next item if needed.

Complexity: Time O(n log n), Space O(n) for a sorting-based implementation.

Q51. What is Job Sequencing with Deadlines?

Ans. It schedules jobs within their deadlines to maximize total profit, typically assuming each job takes one time unit.

Q52. What is Activity Selection?

Ans. Activity Selection chooses the maximum number of mutually compatible activities using their start and finish times.

Q53. Which greedy rule is used in Activity Selection?

Ans. Always select the compatible activity with the earliest finishing time.

Dynamic Programming

Q54. What is Dynamic Programming?

Ans. Dynamic Programming solves overlapping subproblems and stores their results to avoid repeated calculations.

Q55. What are the two important properties of DP?

Ans. Optimal Substructure and Overlapping Subproblems.

Q56. What is Memoization?

Ans. Memoization is a top-down DP technique that stores results of recursive calls.

Q57. What is Tabulation?

Ans. Tabulation is a bottom-up DP technique that solves smaller problems first using a table.

Q58. What is 0/1 Knapsack?

Ans. 0/1 Knapsack selects or rejects each item completely to maximize profit without exceeding capacity.

Complexity: Time O(nW), Space O(W) with optimized DP.

Q59. What is Matrix Chain Multiplication?

Ans. Matrix Chain Multiplication finds the best parenthesization of matrices to minimize scalar multiplications.

Complexity: Time $O(n^3)$, Space $O(n^2)$.

Q60. What is OBST?

Ans. Optimal Binary Search Tree (OBST) arranges keys to minimize expected search cost using their access frequencies.

Q61. What is the complexity of OBST?

Ans. Standard DP: Time $O(n^3)$, Space $O(n^2)$.

Q62. How is DP used for Binomial Coefficients?

Ans. It uses $C(n,k) = C(n-1,k-1) + C(n-1,k)$ and stores previous results.

Complexity: Time O(nk), Space O(k) with optimized DP.

Q63. Difference between Greedy and DP?

Ans. Greedy makes locally optimal choices. DP stores and compares subproblem results to obtain an optimal solution.

Unit IV – Backtracking and Branch and Bound

Backtracking

Q64. What is Backtracking?

Ans. Backtracking tries possible solutions step by step and goes back when a choice cannot lead to a valid solution.

Q65. What is the principle of Backtracking?

Ans. Choose, Explore, and Undo. If a choice is invalid, undo it and try another.

Q66. What is a State Space Tree?

Ans. A State Space Tree represents all possible states and choices while solving a problem.

Q67. What is a Promising Node?

Ans. A promising node is a node that may lead to a valid solution.

Q68. What is the N-Queens Problem?

Ans. Place N queens on an N×N chessboard so that no two queens attack each other.

Q69. How does N-Queens Backtracking work?

Ans.

1. Place a queen in a safe column.
2. Move to the next row.
3. Check column and diagonals.
4. If no safe position exists, backtrack.

Q70. What is the complexity of N-Queens?

Ans. Time O(N!) as a common upper bound for row-wise backtracking; auxiliary space O(N) with optimized tracking, excluding stored solutions.

Q71. What is Graph Coloring?

Ans. Graph Coloring assigns colors to vertices so that no two adjacent vertices have the same color.

Q72. What is the M-Coloring Problem?

Ans. It checks whether a graph can be colored using at most M colors without coloring adjacent vertices the same.

Q73. What is Sum of Subsets?

Ans. Sum of Subsets finds subsets whose elements add up to a given target sum.

Branch and Bound

Q74. What is Branch and Bound?

Ans. Branch and Bound solves optimization problems by exploring possible solutions and pruning branches that cannot improve the best result.

Q75. What is Branching?

Ans. Branching divides a problem into smaller subproblems represented by child nodes.

Q76. What is Bounding?

Ans. Bounding estimates the best possible result from a node to decide whether it should be explored.

Q77. What is FIFO Branch and Bound?

Ans. FIFO explores nodes in First-In-First-Out order using a queue.

Q78. What is LIFO Branch and Bound?

Ans. LIFO explores the most recently generated node first using a stack.

Q79. What is LC Branch and Bound?

Ans. Least Cost (LC) selects the live node with the lowest estimated cost using a priority queue.

Q80. What is a Live Node?

Ans. A live node is a generated node that has not yet been expanded or discarded.

Q81. What is an E-Node?

Ans. An E-Node is the live node currently selected for expansion.

Q82. What is TSP?

Ans. Travelling Salesman Problem (TSP) finds the minimum-cost tour that visits every city exactly once and returns to the starting city.

Q83. How does Branch and Bound solve TSP?

Ans.

1. Generate possible tours.
2. Calculate lower cost bounds.
3. Explore promising nodes.
4. Prune nodes that cannot improve the best tour.

Q84. What is the worst-case complexity of TSP using Branch and Bound?

Ans. Exponential/factorial time, O(n!) in the worst case; memory depends on the search strategy.

Q85. Difference between Backtracking and Branch and Bound?

Ans. Backtracking prunes invalid or infeasible solutions. Branch and Bound prunes solutions that cannot improve the current best value.

Unit V – Amortized Analysis

Amortized Analysis Methods

Q86. What is Amortized Analysis?

Ans. Amortized Analysis finds the average cost per operation over a sequence of operations, even when some operations are expensive.

Q87. What is Aggregate Analysis?

Ans. Aggregate Analysis calculates the total cost of n operations and divides it by n.

Q88. What is the Accounting Method?

Ans. Accounting Method assigns an artificial cost to operations and saves extra credit to pay for future expensive operations.

Q89. What is the Potential Method?

Ans. Potential Method uses a potential function to represent stored work or credit in a data structure.

Q90. What is a Potential Function?

Ans. A Potential Function Φ represents stored credit that can help pay for future operations.

Q91. What is Amortized Cost?

Ans. It is the actual operation cost plus the change in potential: Amortized Cost = Actual Cost + $\Delta\Phi$.

Q92. Explain Binary Counter Amortized Analysis.

Ans. A binary counter may flip many bits during one increment, but over n increments the total flips are O(n).

Complexity: O(1) amortized time per increment.

Q93. What is Stack Amortized Analysis?

Ans. For PUSH, POP, and MULTIPOP, each element can be pushed and popped at most once per push, giving O(1) amortized cost per operation.

Q94. What is Time-Space Tradeoff?

Ans. Time-Space Tradeoff means using more memory to reduce execution time or using less memory at the cost of more time.

Advanced Topics

Q95. What is a Tractable Problem?

Ans. A tractable problem is generally a problem that can be solved in polynomial time.

Q96. What is an Intractable Problem?

Ans. An intractable problem is one for which no efficient polynomial-time algorithm is known or possible under the relevant model.

Q97. What is a Randomized Algorithm?

Ans. A Randomized Algorithm uses random choices during execution to influence its behavior or performance.

Q98. What is an Approximation Algorithm?

Ans. An Approximation Algorithm finds a near-optimal solution for an optimization problem, often with a proven quality guarantee.

Q99. What is an Embedded Algorithm?

Ans. An Embedded Algorithm is designed for systems with limited memory, processing power, and energy.

Q100. What is Power-Optimized Scheduling?

Ans. Power-Optimized Scheduling arranges tasks to reduce energy consumption while satisfying timing requirements.

Q101. Why is Sorting Important in Embedded Systems?

Ans. Sorting helps organize data efficiently while considering limited memory, execution time, and power.

Q102. What is the Cutting Stock Problem?

Ans. Cutting Stock minimizes material waste by deciding how to cut large pieces into smaller required sizes.

Unit VI – Multithreaded and Distributed Algorithms

Multithreaded Algorithms

Q103. What is Multithreading?

Ans. Multithreading allows multiple threads to execute tasks concurrently within the same process.

Q104. What is a Thread?

Ans. A thread is the smallest unit of execution within a process and shares the process's memory.

Q105. What is a Multithreaded Algorithm?

Ans. A Multithreaded Algorithm divides work into tasks that can run concurrently, potentially improving performance.

Q106. What is Parallelism?

Ans. Parallelism means executing multiple tasks at the same time using multiple processing units.

Q107. What is Work in Parallel Algorithms?

Ans. Work (T_1) is the total time required to execute all operations on a single processor.

Q108. What is Span?

Ans. Span ($T\infty$) is the execution time of the longest dependency path with unlimited processors.

Q109. What is Speedup?

Ans. Speedup = T_1/T_p, where T_1 is single-processor time and T_p is execution time on p processors.

Q110. What is Parallel Efficiency?

Ans. Efficiency = Speedup/p, where p is the number of processors.

Q111. What is a Race Condition?

Ans. A Race Condition occurs when multiple threads access shared data and the result depends on their execution order.

Q112. How can we avoid Race Conditions?

Ans. By using locks, synchronization, atomic operations, or thread-safe data structures.

Q113. What is a Parallel Loop?

Ans. A Parallel Loop executes independent iterations concurrently using multiple threads.

Q114. What is Multithreaded Matrix Multiplication?

Ans. It calculates independent matrix elements or blocks simultaneously using multiple threads.

Q115. What is Multithreaded Merge Sort?

Ans. It sorts different parts of an array concurrently and then merges the sorted results.

Complexity: Work O(n log n), auxiliary space O(n) for standard merge sort; parallel time depends on implementation and processors.

Distributed Algorithms

Q116. What is a Distributed Algorithm?

Ans. A Distributed Algorithm runs across multiple connected computers that communicate by passing messages.

Q117. Difference between Multithreaded and Distributed Algorithms?

Ans. Multithreaded algorithms generally share process memory. Distributed algorithms run on separate computers and communicate through messages.

Q118. What is Distributed BFS?

Ans. Distributed BFS explores graph vertices level by level using multiple connected processing nodes.

Q119. What is Distributed MST?

Ans. Distributed MST constructs a minimum spanning tree using communication and coordination between network nodes.

String Matching

Q120. What is String Matching?

Ans. String Matching finds occurrences of a pattern inside a larger text.

Q121. What is Naive String Matching?

Ans. Naive String Matching compares the pattern at every possible position in the text.

Complexity: Worst-case time O(nm), auxiliary space O(1).

Q122. What is Rabin-Karp Algorithm?

Ans. Rabin-Karp uses hashing to find matching patterns efficiently and verifies possible matches.

Q123. Explain Rabin-Karp Algorithm.

Ans.

1. Calculate the pattern hash.
2. Calculate the current text-window hash.
3. Compare hashes and verify matches.
4. Slide the window using a rolling hash.

Q124. What is Rolling Hash?

Ans. Rolling Hash efficiently updates the hash value when a fixed-size window moves by one position.

Q125. What is Hash Collision?

Ans. A Hash Collision occurs when different strings produce the same hash value.

Q126. What is the complexity of Rabin-Karp?

Ans. Expected time O(n + m), worst-case O(nm), auxiliary space O(1) for a standard single-pattern rolling-hash implementation.

Group A – DAA Practical Viva Questions

All 6 practical assignments from your provided syllabus

Practical 1 – Recursive and Non-Recursive Fibonacci

Q127. What is Fibonacci Series?

Ans. Fibonacci Series is a sequence where each number is the sum of the previous two numbers. Example: 0, 1, 1, 2, 3, 5, 8.

Q128. What is the Fibonacci Formula?

Ans. $F(n) = F(n-1) + F(n-2)$, where $F(0) = 0$ and $F(1) = 1$.

Q129. What is Recursion?

Ans. Recursion is a technique where a function calls itself to solve smaller versions of the same problem.

Q130. What is a Base Case?

Ans. A Base Case is the stopping condition that prevents infinite recursive calls.

Q131. What is the difference between Recursive and Iterative Fibonacci?

Ans. Recursive Fibonacci uses function calls, while Iterative Fibonacci uses loops.

Q132. What is the complexity of Recursive Fibonacci?

Ans. Time $O(2^n)$, Space O(n) for simple recursion without memoization.

Q133. What is the complexity of Iterative Fibonacci?

Ans. Time O(n), Space O(1) when only the previous two values are stored.

Q134. Which Fibonacci approach is better?

Ans. Iterative Fibonacci is generally better because it avoids repeated calculations and uses constant extra memory.

Practical 2 – Huffman Encoding

Q135. What is Huffman Encoding?

Ans. Huffman Encoding is a greedy algorithm used for lossless data compression.

Q136. Which data structure is used in Huffman Coding?

Ans. A Min-Heap (Priority Queue) is used to select the two nodes with minimum frequency.

Q137. Why is Huffman Coding a Greedy Algorithm?

Ans. It repeatedly selects the two lowest-frequency nodes to construct an optimal prefix code.

Q138. What is the purpose of Huffman Coding?

Ans. To reduce data size by assigning shorter codes to frequently occurring characters.

Q139. Is Huffman Coding Lossy or Lossless?

Ans. Lossless, because the original data can be reconstructed exactly.

Q140. What is the complexity of Huffman Coding?

Ans. Time O(n log n), Space O(n) for n distinct symbols using a min-heap.

Practical 3 – Fractional Knapsack

Q141. Which algorithm is used for Fractional Knapsack?

Ans. The Greedy Algorithm.

Q142. Why do we calculate the Profit-to-Weight Ratio?

Ans. To select items that provide the maximum profit per unit weight.

Q143. Can we divide items in Fractional Knapsack?

Ans. Yes, items can be divided into fractions.

Q144. Which sorting order is used?

Ans. Descending order of profit/weight ratio.

Q145. What is the complexity of Fractional Knapsack?

Ans. Time O(n log n) due to sorting; Space O(n) in a typical implementation.

Practical 4 – 0/1 Knapsack

Q146. Which techniques solve 0/1 Knapsack?

Ans. Dynamic Programming and Branch and Bound.

Q147. Why is 0/1 Knapsack called 0/1?

Ans. Each item has two choices: 0 means reject and 1 means select.

Q148. Can Greedy always solve 0/1 Knapsack optimally?

Ans. No, greedy choices do not always produce the optimal solution for 0/1 Knapsack.

Q149. What is the DP State in 0/1 Knapsack?

Ans. dp[i][w] represents the maximum profit using the first i items with capacity w.

Q150. What is the complexity of DP-based 0/1 Knapsack?

Ans. Time O(nW), Space O(nW) for table-based DP, reducible to O(W) space.

Practical 5 – N-Queens Using Backtracking

Q151. Which algorithm is used for N-Queens?

Ans. Backtracking.

Q152. What are the conditions for placing a Queen?

Ans. No two queens can share the same row, column, or diagonal.

Q153. What happens when no safe position is available?

Ans. The algorithm backtracks, removes the previous queen, and tries another position.

Q154. What is special about your N-Queens assignment?

Ans. The first queen is already placed, and we must place the remaining queens without conflicts.

Q155. Can N-Queens have multiple solutions?

Ans. Yes, depending on N and the fixed first queen, there may be multiple solutions or no solution.

Q156. What is the complexity of N-Queens?

Ans. Time O(N!) as a common upper bound for row-wise search; optimized auxiliary space O(N), excluding the board and stored solutions.

Practical 6 – Deterministic and Randomized Quick Sort

Q157. What is Quick Sort?

Ans. Quick Sort is a divide-and-conquer sorting algorithm that partitions elements around a pivot.

Q158. What is a Pivot?

Ans. A pivot is the selected element used to divide the array into partitions.

Q159. What is Deterministic Quick Sort?

Ans. Deterministic Quick Sort selects the pivot using a fixed rule, such as the first or last element.

Q160. What is Randomized Quick Sort?

Ans. Randomized Quick Sort selects the pivot randomly to reduce the chance of consistently unbalanced partitions.

Q161. What is Partitioning?

Ans. Partitioning rearranges array elements so smaller values come before the pivot and larger values come after it.

Q162. What is the complexity of Quick Sort?

Ans. Average $O(n \log n)$, Worst $O(n^2)$; average recursive stack space $O(\log n)$.

Q163. Which is better: Deterministic or Randomized Quick Sort?

Ans. Randomized Quick Sort generally gives more reliable expected performance across different input arrangements.

Q164. Is Quick Sort Stable?

Ans. Standard in-place Quick Sort is not stable, because equal elements may change their original relative order.

Last-Minute Practical Complexity Revision

Practical	Time Complexity	Auxiliary Space
Iterative Fibonacci	$O(n)$	$O(1)$
Recursive Fibonacci	$O(2^n)$	$O(n)$
Huffman Coding	$O(n \log n)$	$O(n)$
Fractional Knapsack	$O(n \log n)$	$O(n)$
0/1 Knapsack DP	$O(nW)$	$O(W)$ optimized
N-Queens	$O(N!)$ upper bound	$O(N)$ optimized
Deterministic Quick Sort	$O(n \log n)$ average	$O(\log n)$ average
Randomized Quick Sort	$O(n \log n)$ expected	$O(\log n)$ expected